

X-Planner: Event-Structured Task Planning for Embodied Intelligence

X Square Robot Team

Task planning bridges high-level instructions and executable behavior in long-horizon manipulation, yet modern Vision–Language–Action (VLA) systems often leave this intermediate structure implicit. Existing chain-of-thought (CoT) planners also tend to rely on coarse task-level annotations or serialize long reasoning traces token by token. We present X-PLANNER, a planning front-end that addresses both the supervision and representation of embodied reasoning. Our planning data combine Ego, UMI, and teleoperation under a hierarchy granularity with source-dependent annotation depth. Takeover-time annotations and human-designed failures supervise ongoing error recognition. On the model side, a shared VLM backbone exposes two event-structured plan forms: a discrete interface that emits interpretable event states and a latent interface that relays continuous CoT states across staggered Transformer depths through *Staircase Decoding*. A frozen latent-to-text reconstruction objective provides a semantic anchor for the latent representation. Offline two-step planning evaluation places X-PLANNER second among four evaluated models on both BERTScore-F1 and a judge-based Overall score. In real-robot experiments, respectively, outperforming the evaluated baselines. These results characterize planning-text quality and downstream execution.

Date: September 16, 2026

Code: <https://github.com/X-Square-Robot/Xplanner>

1 Introduction

Embodied foundation models have advanced rapidly by mapping visual observations and natural-language instructions to executable actions (30, 15, 3, 11, 2, 5). This direct observation-to-action paradigm is effective for short, well-specified skills. Long-horizon tasks, however, require an agent to select and order grounded sub-goals before issuing low-level commands. Many Vision–Language–Action (VLA) models leave this intermediate structure implicit. Bridging high-level intent and low-level control is therefore a planning problem as well as a multimodal-representation problem. Recent systems begin to address it by introducing chain-of-thought (CoT) reasoning before action generation (26, 28).

Despite this progress, embodied task planning remains fragmented across data construction, planning granularity, and decoding strategy. We focus on three gaps.

<https://openai.com/index/gpt-6-astra>

The data gap. Embodied planners are often trained with task-level descriptions or hand-written sub-goal lists for a fixed suite. Such labels rarely specify *where* one sub-goal ends and the next begins. A single episode caption can obscure the internal event structure of a demonstration: regrasps, failed contacts, retries, and small pose corrections may all disappear behind a label such as “pick up the object” (21). Planning supervision must therefore capture event boundaries and execution errors as they arise. Failure examples collected from a single policy may also overrepresent that policy’s characteristic mistakes.

The granularity gap. Reasoning-based planners also tend to commit to a single, externally imposed granularity. Some emit text at a fixed semantic level, whereas others forecast sub-goal images or optical flow (28). We instead adopt the *semantic event* as the planning unit: a temporally coherent span of executable behavior, such as reaching, grasping, lifting, or placing, whose boundary follows a change in behavior. In contrast, a fixed-length chunk may split one behavior or merge several behaviors into a single target. Action-grounded

events provide a useful middle ground: each plan step is meaningful in language, observable in video, and realizable through control.

The latency gap. Finally, conventional CoT planners decode reasoning autoregressively, one token at a time. Over a long rollout, this serial dependency can repeatedly process overlapping visual–language context and delay the next planning handoff (8, 29). Long explicit traces are also vulnerable to error propagation. Latent-CoT methods replace discrete tokens with continuous states, but many retain a serial dependency between latent steps. A practical planner should reduce this serial critical path while exposing a representation that the execution stack can consume at a well-defined handoff.

We present X-PLANNER, a task-planning front-end that addresses these three gaps jointly. Given a high-level instruction and the current multi-view observation, X-PLANNER produces an event-structured representation that conditions a downstream world-action model. The architecture is detailed in Sec. 4 and Fig. 4. Our contributions are threefold.

1. **Multisource, hierarchical planning supervision.** We combine Ego, UMI, and teleoperation under an L3 Task/L2 Subtask/L1 Action/L0 Segment hierarchy. Ego and UMI retain L1–L3; teleoperation supports all four levels. Annotated takeover times supervise ongoing error recognition, while human-designed failure demonstrations supplement policy-collected errors to reduce dependence on a single policy’s failure distribution. A 1,500-episode analysis subset characterizes semantic and temporal coverage.
2. **Staircase Decoding with discrete and latent plan forms.** A shared VLM backbone produces either explicit event states or continuous CoT states relayed across staggered Transformer depths. The latent form avoids token-by-token serialization within the planner, while a frozen latent-to-text objective encourages the compact states to retain plan semantics.
3. **Offline planning and real-robot evaluation.** Offline two-step text evaluation measures semantic matching and overall plan quality: X-PLANNER scores above Qwen and Doubao and below kimi3 on both reported metrics. Coupling X-PLANNER to a world-action backbone yields the highest average Task Progress among the evaluated systems on the reported reasoning and generalization suites.

Together, these components frame embodied planning around action-grounded events, reproducible supervision, and complementary explicit and latent interfaces. The reported experiments evaluate planning text offline and the complete event-mode system on real robots; controlled comparisons of the two plan forms are left for future work.

2 Related Work

2.1 Reasoning and Planning in VLA Models

Vision–Language–Action models extend pretrained vision–language models with interfaces that map observations and instructions to continuous control commands (30, 15, 22, 3, 11, 2, 5). Their semantic priors support generalization across objects, scenes, and instructions, yet the action interface is often trained as a largely reactive observation-to-action map with no explicit representation of task structure (17, 5). A growing line of work introduces CoT reasoning to decompose a task before acting (26, 28, 10). Linguistic approaches emit textual sub-goals or reasoning traces, as in Embodied CoT (26); visual approaches forecast sub-goal images or dense motion cues, as in CoT-VLA (28). These representations trade semantic readability against spatial specificity and generation cost. X-PLANNER instead grounds both its explicit and latent representations in action-aligned semantic events, leaving fine spatiotemporal realization to the downstream world-action model.

2.2 Explicit, Latent, and Parallel Reasoning

A complementary line of work studies *how* reasoning is decoded. Explicit CoT generates discrete tokens sequentially, creating a serial critical path and allowing an early error to affect later steps. Latent-reasoning methods reduce textual serialization by routing CoT through compact continuous states (8, 6, 12, 13, 29). Coconut (8) and LaDiR (12), for example, compress intermediate thoughts into continuous representations; recent embodied variants distill spatiotemporal or world-model foresight for manipulation and driving (18, 19,

1). Many such methods nevertheless retain an autoregressive dependency between latent steps. X-PLANNER’s Staircase Decoding instead relays hidden states across staggered Transformer depths, allowing latent plan states to share lower-layer grounding computation and proceed in parallel through the upper layers.

2.3 Data and Annotation for Embodied Planning

Internet video (20) and egocentric corpora (7) provide visual-dynamics priors. Robot datasets (14, 4) pair demonstrations with control actions. Episode-level task strings alone conceal the sub-goal structure required by a planner. Temporally grounded captioning and atomic-action segmentation (21, 9) motivate finer supervision. Our planning data (Sec. 3) combine Ego, UMI, and teleoperation with source-dependent annotation depth, grounding event order and progress in a shared hierarchy. Takeover-time annotations and human-designed failures extend this supervision to recognizing errors during ongoing execution.

3 Event-Structured Planning Data

3.1 Sources and Hierarchical Annotation

We build planning supervision from egocentric (Ego), UMI, and teleoperated demonstrations (Fig. 1). A shared hierarchy represents **L3 Task** (episode objective), **L2 Subtask** (semantic stage), **L1 Action** (executable primitive), and **L0 Segment** (finest temporal span). Teleoperation data carry all four levels. Ego and UMI retain L1–L3: their faster motions make L0 boundaries difficult to annotate reliably, so Action is their finest labeled level. Available visual and action streams are synchronized and screened for missing observations, timestamp inconsistencies, and unusable records before annotation.

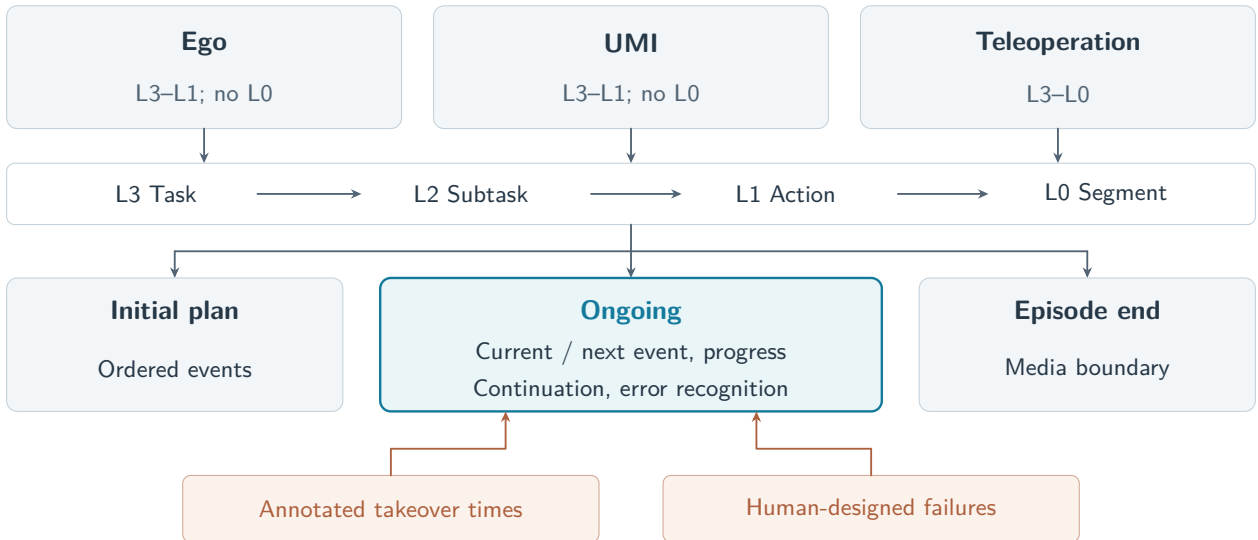


Figure 1 Planning data and supervision. Three data sources share a hierarchy with source-dependent granularity. Takeover timing and human-designed failures provide ongoing error-recognition supervision.

3.2 Failure-Aware Ongoing Supervision

Ongoing supervision must identify execution errors as well as track plan progress. We therefore annotate takeover times in intervention episodes, providing temporal supervision for error recognition during execution. Because these episodes reflect the failure distribution of the collecting policy, we supplement them with human-performed demonstrations of deliberately designed failure modes. These simulated failures broaden error coverage and are intended to reduce dependence on a single policy’s biases. Together with the hierarchy, the annotations support initial-plan, ongoing, and episode-end states; ongoing targets cover current/next events, normalized progress, continuation state, and error recognition. Materialization details are given in Appendix A.

3.3 Coverage and Selection

A deterministic analysis subset retains 1,500 of 1,654 candidate episodes. Removing 154 redundant, frequent combinations preserves all 167 datasets, 30 named task categories plus an unknown label, and the observed tail of semantic and action labels. Fixed-seed tie-breaking makes selection repeatable. These statistics describe the analyzed pool, rather than the full three-source training mixture or its source proportions.

Figure 2A–C shows independent object, capability, and scene shares, dominated by rigid objects (58.2%), scene grounding (39.4%), and office/public scenes (28.5%). The temporal empirical cumulative distribution functions (ECDFs; D–E) have medians of five subtasks and 37.1 s, and 90th percentiles of 15 subtasks and 113.1 s (dashed and dotted guides, respectively). Subtask counts reach 65; episode duration is the maximum video duration across cameras, reaches 822.7 s, and uses a logarithmic axis.

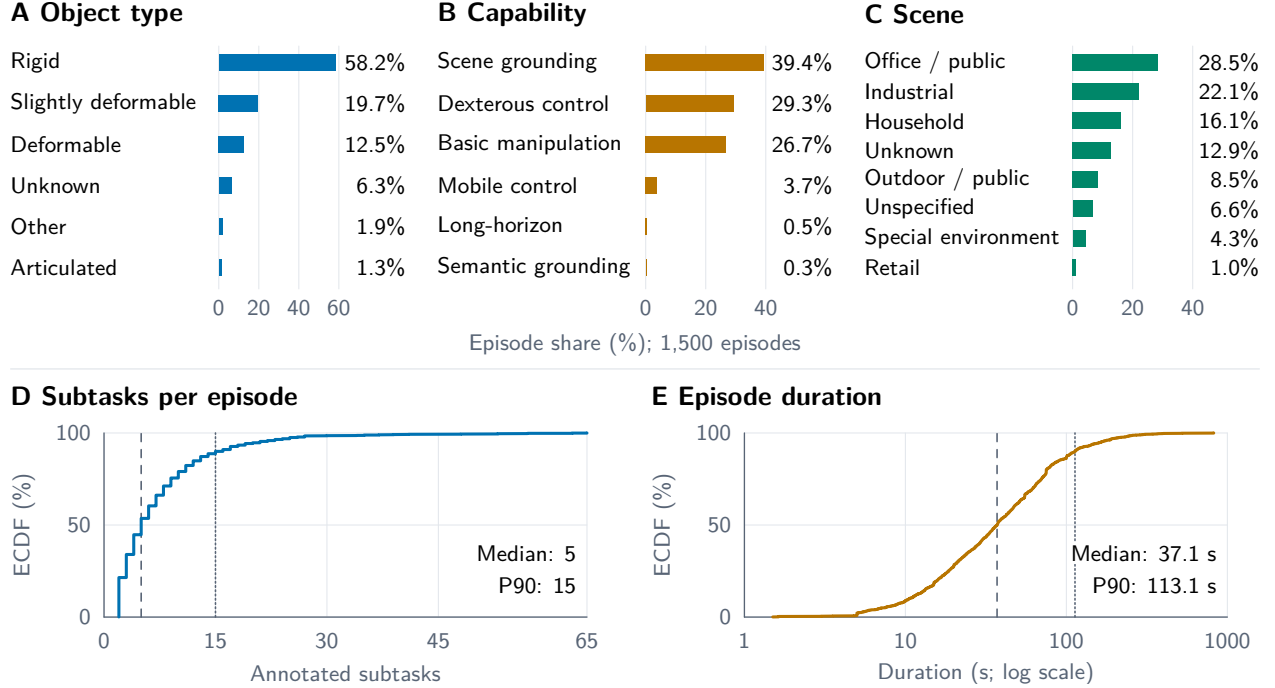


Figure 2 Semantic and temporal coverage of the 1,500-episode analysis subset.

Figure 3a–b characterizes action and task coverage. The manifest contains 41 named atomic-action labels inferred from two ground-truth subtask captions per episode, with one unmatched episode. Move, grasp, and place cover 68.4%, 57.0%, and 40.3% of episodes; pick-and-place, opening/closing containers, and sorting/storage cover 85.8%, 28.8%, and 27.5%. These are multi-label episode shares, not action-occurrence frequencies. All 1,500 episodes remain in the denominator, including 17 with an unknown task label; the complete counts preserve the source taxonomy.

Figure 3c contrasts source-defined action- and subtask-complexity metadata. The action bins 1–3, 4–6, and > 6 account for 58.9%, 30.7%, and 9.3%, respectively. Subtask-complexity shares are 28.2%, 35.3%, and 35.4%; both metadata fields have 1.1% unknown labels. These categories remain separate from measured subtask counts and caption-inferred action types and do not constitute calibrated difficulty scores.

Figure 3D–E reports the effect of deterministic selection as the retained share minus the candidate-pool share. Every displayed shift is negative because the procedure removes redundant frequent combinations instead of reweighting labels: the largest coverage change is for rigid objects (−3.7 percentage points), followed by scene grounding (−2.0 pp), while the source-group and camera-count shifts are at most −0.6 pp. The same pattern appears for atomic actions, with Place (−3.3 pp) and Move (−2.4 pp) changing most and Grasp changing by −1.5 pp. Thus, selection trims dominant modes while keeping the taxonomy and long tail available; these are episode-share shifts, not changes in per-action frequency or calibrated difficulty.

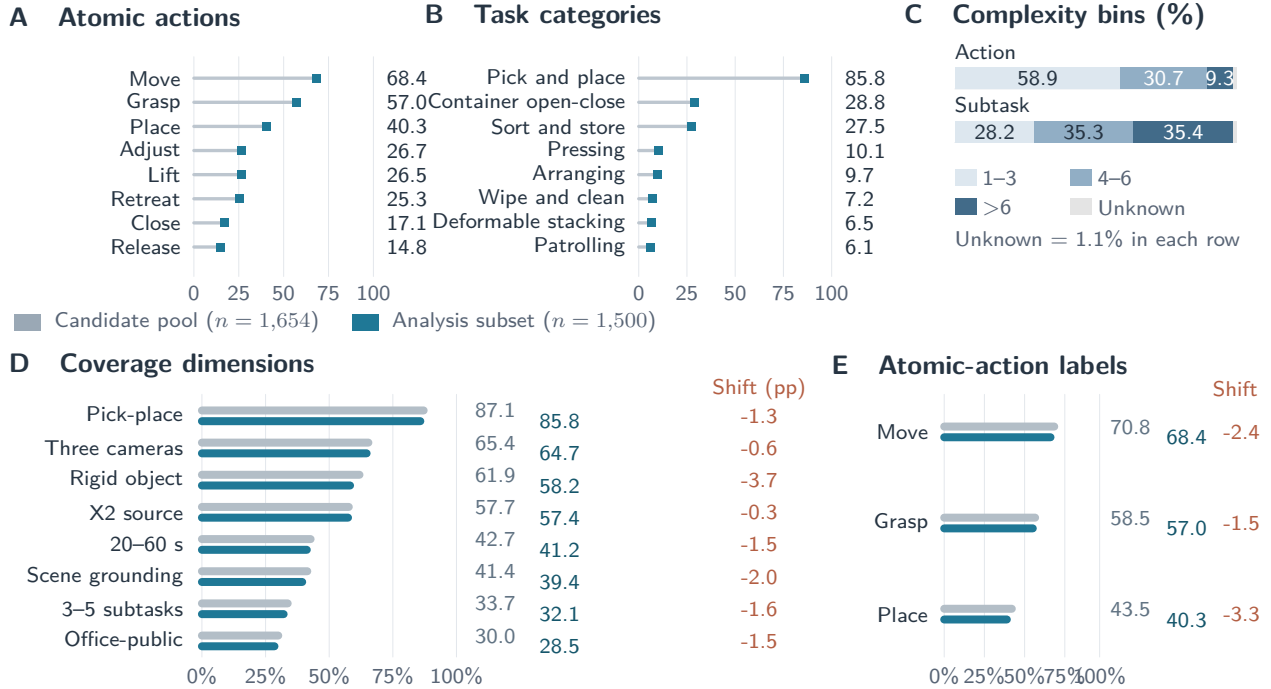


Figure 3 Action/task profiles and selection effects. A–C show episode shares and complexity-bin composition; D–E show retained-minus-candidate shifts (percentage points). All panels use the same 1,500-episode subset and denominator; labels are multi-label where applicable.

4 Method

Given a high-level instruction ℓ , the available visual observations $\{\mathbf{V}_0^{(\nu)}\}_\nu$, and optional history h , X-PLANNER produces an event-structured representation that conditions a downstream world-action model. The view index ν ranges over the views available in each example. A Qwen-series VLM backbone (24, 23) provides the visual-language features used for scene grounding, event decomposition, and progress estimation. As shown in Fig. 4, the architecture exposes the plan in two forms: a discrete event state for interpretable planning and a latent state sequence that avoids token-by-token serialization within the planner.

4.1 Two Plan Forms

Discrete form: structured event states. In the discrete form, the VLM emits a compact event state rather than an isolated caption. At the start of execution, it produces an ordered initial plan. During rollout, its structured response covers normalized progress, the current event, the next event when available, continuation state, and execution-error recognition. An event caption may, for example, read *align the gripper above the red cup*. At the final available media frame, the model emits an episode-end marker. These structured states are inspectable and editable by a human or upstream agent, and their event descriptions provide the text interface to the downstream world-action model. The episode-end marker denotes the recorded-media boundary; it is not, by itself, a certificate of semantic task success.

Latent form: parallel continuous CoT. The discrete form relies on autoregressive token generation. The latent form instead represents the plan as a compact sequence of K_c continuous reasoning states, $\hat{\mathbf{y}}_{1:K_c} = \{\hat{\mathbf{y}}_1, \dots, \hat{\mathbf{y}}_{K_c}\}$. The staircase branch generates these states in parallel and injects them directly into the downstream cross-attention pathway. Both forms share the VLM backbone and event-grounded semantics (Sec. 5); they differ in whether the plan is exposed as text or retained as continuous states.

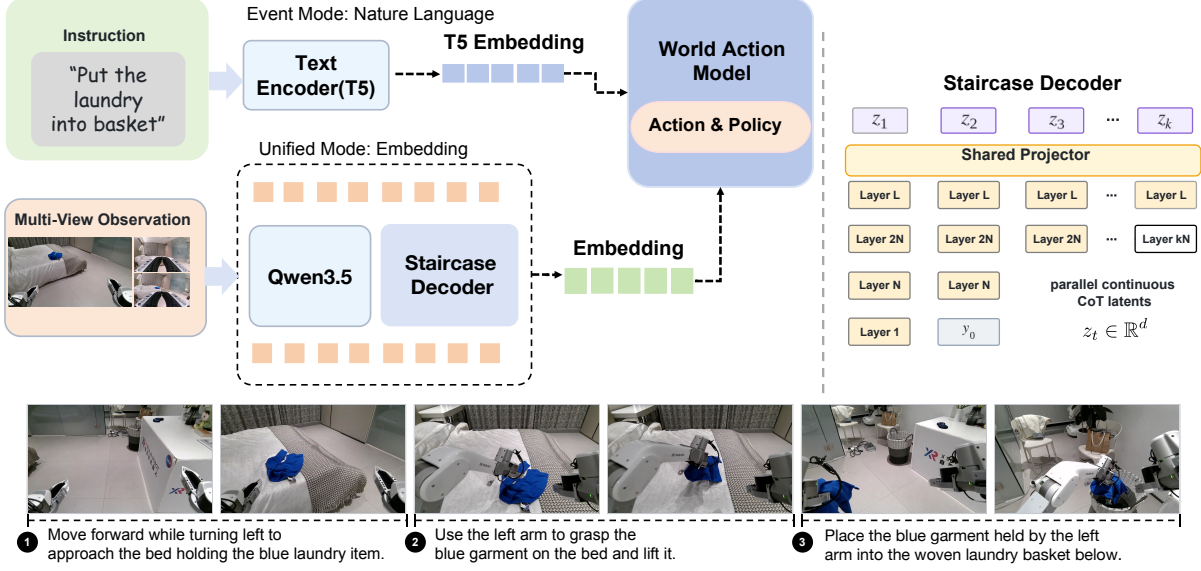


Figure 4 Overview of X-PLANNER. Given a high-level instruction and multi-view observations, the planner exposes an interpretable event-language interface and a continuous conditioning interface to a downstream world-action model. Staircase Decoding constructs latent plan states across staggered Transformer depths; the lower row illustrates an event-level rollout.

4.2 Text Conditioning Interface

Both plan forms use a common conditioning interface. Let $\mathbf{H}_q$ denote the VLM hidden states. A boundary mask $\mathbf{m}_{\text{txt}}$ selects the text-token states, which are projected into the downstream conditioning space:

$$\mathbf{c}_\ell = \mathcal{M}_T(\mathcal{P}_Q(\mathbf{H}_q[\mathbf{m}_{\text{txt}}])), \quad (1)$$

where $\mathcal{P}_Q$ is a learnable projection and $\mathcal{M}_T$ is the downstream backbone’s native text MLP. The resulting sequence enters the world-action cross-attention layers alongside image tokens. The alignment objective in Sec. 5.2 encourages these projected VLM states to match the text-feature geometry expected by the fixed downstream model. This interface allows the planner to add scene-grounded disambiguation, task decomposition, and progress-aware event context without replacing the downstream text pathway.

4.3 Staircase Latent CoT

The latent form is implemented through *Staircase Decoding*. In a conventional autoregressive latent rollout, each reasoning state depends on the previous state and traverses the full Transformer stack. Staircase Decoding shortens this serial path through a depth-parallel schedule.

The reasoning branch is initialized from the fine-tuned VLM and implemented as a lightweight Mixture-of-Transformers (MoT) structure coupled to the frozen backbone. We partition the Transformer at a *relay depth* N_r . The lower layers encode shared visual–language context, whereas the upper layers specialize across reasoning steps. The first latent position traverses the lower layers to produce a relay representation shared by all reasoning positions. The upper blocks then update the latent states in parallel, with an independent causal cache for each position:

$$\hat{\mathbf{y}}_{1:K_c} = \mathcal{F}_{\text{stair}}(x; N_r), \quad (2)$$

where $x = (\{\mathbf{V}_0^{(v)}\}_v, \ell, h)$ and N_r controls how much grounding computation is shared before the reasoning paths diverge. Relative to a fully autoregressive rollout, the schedule reuses lower-layer visual–language features

across reasoning states (Fig. 4). The resulting latents condition the downstream cross-attention pathway without discrete token sampling. This architectural reduction in serial computation does not, by itself, establish lower end-to-end control latency, which also depends on the surrounding execution stack.

4.4 Frozen Latent-to-Text Reconstruction

A latent plan is useful only if its compact states retain the intended event semantics. We encourage this property with a *frozen latent-to-text reconstruction* objective (12, 29), rather than by matching a particular sequence of autoregressive hidden states. A prefix projector $\mathcal{P}_{\text{pref}}$ maps $\hat{y}_{1:K_c}$ to a soft prefix $\mathbf{z}_{1:K_c} = \mathcal{P}_{\text{pref}}(\hat{y}_{1:K_c})$ in the embedding space of a lightweight frozen language model. Conditioned on this prefix, the language model reconstructs the corresponding textual CoT trace:

$$P_\phi(r_{1:M_r} \mid \mathbf{z}_{1:K_c}), \quad \mathcal{L}_{\text{CoT}} = - \sum_{m=1}^{M_r} \log P_\phi(r_m \mid \mathbf{z}_{1:K_c}, r_{<m}). \quad (3)$$

Only the staircase reasoning branch and $\mathcal{P}_{\text{pref}}$ are optimized; the reconstruction model remains fixed. Reconstructing the trace encourages the latent sequence to preserve high-level semantics without copying a specific token-level hidden trajectory. The frozen decoder can also render the latent plan as text for qualitative inspection.

4.5 Inference: Coupling X-PLANNER to a World-Action Model

The two plan forms define corresponding deployment modes. In **event mode**, X-PLANNER emits an initial plan at startup and updates its structured event state as new observations arrive. Event descriptions condition the world-action model, and ongoing error recognition supports execution monitoring. The execution system determines when to refresh or commit the state. In **unified mode**, the staircase decoder produces K_c continuous CoT states that condition fixed-length action-chunk inference through the same cross-attention interface. The common interface therefore supports either readable event plans or continuous latent conditioning.

5 Training

Each training example combines the available visual observations, a task instruction, optional history, and a complete compact JSON target. The discrete interface is trained on this structured response, while feature alignment and staircase reconstruction introduce separate model-side objectives. The downstream world-action model remains fixed throughout.

5.1 Structured Plan-State Supervision

The data pipeline materializes three event-state categories before optimization (Table 1). Supervision uses only the annotation levels supported by each source: Ego and UMI provide L1–L3, while teleoperation additionally provides L0. For failure examples, annotated takeover times and human-designed failures supervise error recognition during ongoing execution. Appendix A details the supervision profiles; Appendix B describes context variants and snapshot construction.

Table 1 Structured states used for discrete planning supervision. Annotation availability and materialization details are given in Appendix A.

State	Visual anchor	Supervised assistant state
Initial plan	Execution start	Ordered initial event plan
Ongoing	During execution	Current/next events, normalized progress, continuation state, and error recognition on annotated failure examples
Episode end	Last available media frame	Final event state and an episode-end marker

Let $x = (\{\mathbf{V}^{(v)}\}_v, \ell, h)$ denote the observations, instruction, and optional history, and let $y_{1:M}$ denote the complete assistant JSON target. We supervise every assistant token autoregressively, rather than optimizing only the next-event caption or regressing progress as an independent scalar:

$$\mathcal{L}_{\text{SFT}} = - \sum_{m \in \mathcal{A}} \log p_{\theta}(y_m \mid x, y_{<m}), \quad (4)$$

where $\mathcal{A}$ indexes assistant positions. Captions, progress, continuation state, and annotated errors are learned jointly within the structured response. Missing annotation levels contribute no supervision; in particular, the absence of L0 labels in Ego and UMI is not a negative Segment label.

5.2 Feature Alignment for Text Conditioning

Separately from the JSON supervision, we align the planner-side conditioning path with the downstream model’s original text-feature space:

$$\mathcal{L}_{\text{align}} = \lambda_{\text{align}} \|\mathbf{c}_{\ell}^{\text{VLM}} - \mathbf{c}_{\ell}^{\text{ref}}\|^2, \quad (5)$$

where $\mathbf{c}_{\ell}^{\text{ref}}$ is the original text encoder’s representation of the same instruction. This compatibility objective encourages the planner representation to remain in the geometry expected by the fixed downstream interface. It is a model-side loss, not an alternative assistant target.

5.3 Staircase Distillation

The staircase module of Sec. 4.3 is a lightweight MoT branch coupled to the frozen VLM backbone. Initializing the branch from pretrained VLM weights transfers the backbone’s visual–language representations to the parallel latent rollout. Given the multimodal input, the branch produces $\hat{y}_{1:K_c}$, which serves as the implicit plan supplied to downstream cross-attention.

For supervision, the prefix projector $\mathcal{P}_{\text{pref}}$ maps the latent sequence to a soft prefix $\mathbf{z}_{1:K_c}$ in the embedding space of a frozen lightweight language model. The language model then reconstructs the textual CoT trace autoregressively:

$$P_{\phi}(r_{1:M_r} \mid \mathbf{z}_{1:K_c}). \quad (6)$$

The latent-to-text objective $\mathcal{L}_{\text{CoT}}$ from Sec. 4.4 updates only the staircase branch and $\mathcal{P}_{\text{pref}}$; all other components remain frozen. The targets $r_{1:M_r}$ are annotation-derived traces that share the event semantics of the discrete form. This reconstruction stage is distinct from compact-JSON SFT: the two paths share the backbone and semantic grounding, but use different targets and readouts.

Scope of the objectives. The structured assistant response defines the SFT target, feature alignment regularizes the downstream interface, and latent reconstruction supervises the staircase representation. We report these roles separately because the current system-level experiments do not isolate their individual causal contributions.

6 Experiments

We evaluate X-PLANNER at two complementary levels: the semantic and overall quality of two-step planning text, and the physical behavior of a X-PLANNER-conditioned world-action system. The real-robot evaluation covers reasoning beyond single-object motion and generalization under clutter and changing instructions.

6.1 Offline Two-Step Planning Evaluation

The offline comparison evaluates two-step planning predictions against reference annotations. We report two complementary metrics: semantic matching of the predicted text and the overall quality of the proposed plan.

Metrics. *BERTScore-F1* (27) measures semantic similarity between the predicted planning text and the reference annotations using contextual token representations. We report the F1 score without baseline rescaling. It measures text-level semantic agreement and should not be interpreted as a task-success rate. *Overall* is a composite quality rating assigned by *Doubao Judge*. Given the task description, reference answer, and model prediction, the judge considers accuracy, continuity, and the overall reasonableness of the planning result. Higher values are better for both metrics.

Table 2 Offline two-step planning quality. BERTScore-F1 is reported without baseline rescaling; Overall is the composite rating from Doubao Judge. Bold indicates the highest value in each column.

Model	BERTScore-F1 $\uparrow$	Overall $\uparrow$
Qwen	0.8878	1.367
Doubao	0.8846	1.366
kimi3	0.9108	1.434
X-PLANNER (ours)	0.9011	1.411

Results. Table 2 shows that X-PLANNER ranks second among the four evaluated models on both metrics. Its BERTScore-F1 of 0.9011 exceeds Qwen and Doubao by 0.0133 and 0.0165, respectively, while remaining 0.0097 below kimi3. The Overall rating follows the same ordering: X-PLANNER scores 1.411, compared with 1.367 for Qwen, 1.366 for Doubao, and 1.434 for kimi3. The two metrics thus give a consistent ranking in this comparison, with X-PLANNER producing stronger semantic matches and higher judge-rated plan quality than Qwen and Doubao. These aggregate scores characterize offline planning text; they do not establish statistical significance or physical execution success.

6.2 Real-Robot Evaluation

The real-robot comparison uses matched physical task definitions, language instructions, multi-view observations, scene-randomization protocols, and scoring rubrics.

Setup and metric. We conduct the experiments on tabletop bimanual arms using synchronized multi-view observations. The primary metric is *Task Progress*, a dense 0–100 score that credits partial completion according to task-specific rubrics. Unlike binary success, it captures partial grounding and execution progress. The evaluated X-PLANNER system operates in *event mode*: given the task instruction and current observation, the planner produces a next-event description that conditions the world-action backbone. U-SCRATCH is a fixed-length instruction-to-action system trained from scratch with the same real-robot supervision. We treat it as a system-level reference rather than an intervention that isolates a particular X-PLANNER component. Where deployable, we additionally report $\pi_{0.5}$ (11), DreamZero (25), and LingBot-VA (16).

Figure 5 consolidates the reported suite-level means in a single visual scale. Values are annotated directly to avoid requiring visual interpolation. LingBot-VA was not available in the Generalization setting and is marked N/A. The available records contain neither trial-level scores nor a reported uncertainty statistic, so the figure intentionally shows no error bars; it should be read as a comparison of reported means rather than a variance or significance analysis.

6.2.1 Reasoning Manipulation

The Reasoning Manipulation suite contains Sort Headphone, Classify Items as Shape, Press Button in Order, Pair Up Items, and Pick Fruits into Basket. Collectively, these tasks test category grounding, relational matching, ordered execution, and instruction-conditioned selection. Because each scene admits several plausible actions, the policy must identify the intended object, category, relation, or sequence before acting.

In Fig. 5A and Table 3, the X-PLANNER-conditioned policy achieves the highest average Task Progress, 71.60. The corresponding averages are 59.50 for U-SCRATCH, 56.40 for $\pi_{0.5}$, 32.70 for DreamZero, and 31.60 for LingBot-VA. Thus, X-PLANNER exceeds the strongest reported baseline in this suite, U-SCRATCH, by 12.10 percentage points.

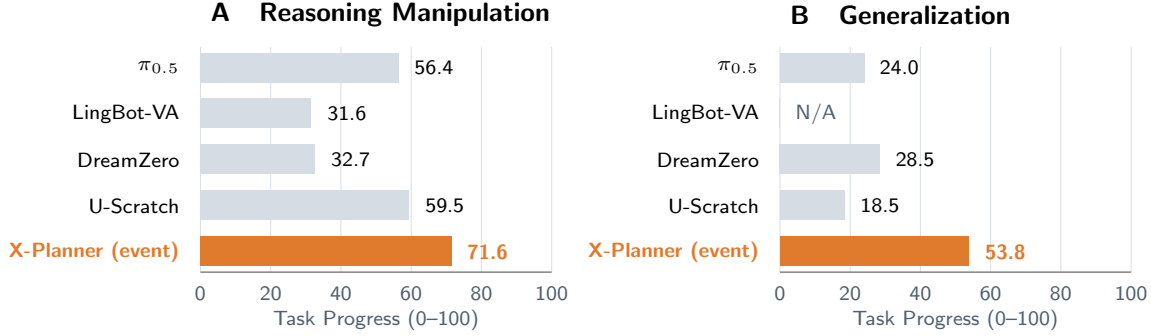


Figure 5 Suite-level real-robot results. Average Task Progress for (A) Reasoning Manipulation and (B) Generalization; higher is better. Each bar is the reported suite mean, and the value is printed at its endpoint. LingBot-VA is marked N/A in panel B because no result is available for that setting. Error bars are omitted because trial-level variation and uncertainty estimates were not provided.

Table 3 Task-level Reasoning Manipulation results. Task Progress (%); higher is better. Task values are rounded chart readouts; Average uses the separately reported suite mean.

Model	Sort Headphone	Classify Items as Shape	Press Button in Order	Pair Up Items	Pick Fruits into Basket	Average
$\pi_{0.5}$	41%	55%	31%	77%	78%	56.40%
LingBot-VA	56%	30%	8%	4%	60%	31.60%
DreamZero	66%	36%	3%	13%	45%	32.70%
U-SCRATCH	84%	82%	18%	43%	70%	59.50%
X-PLANNER	84%	78%	64%	36%	96%	71.60%

Table 3 shows stronger Task Progress for X-PLANNER on ordered button pressing and instruction-conditioned fruit selection, with the same rounded score as U-SCRATCH on headphone sorting. U-SCRATCH scores higher on shape classification, while $\pi_{0.5}$ performs best on item pairing. These task-level patterns are descriptive; the suite-level comparison uses the separately reported means.

6.2.2 Generalization

The Generalization suite tests instruction-conditioned transfer in a shared, cluttered tabletop scene. Figure 5B reports average Task Progress of 53.75 for X-PLANNER, 28.50 for DreamZero, 24.00 for $\pi_{0.5}$, and 18.50 for U-SCRATCH. The margin over the strongest available baseline, DreamZero, is 25.25 percentage points. LingBot-VA is excluded because no result is available for this suite.

6.3 Interpreting the Results

The offline text metrics and real-robot Task Progress address different questions: whether the planner produces semantically appropriate, coherent text, and whether the conditioned system makes physical task progress. X-PLANNER ranks second on both offline metrics and achieves the highest reported mean Task Progress on both robot suites. The robot results evaluate the complete event-mode system; they do not separately estimate the contribution of error-supervision data, the discrete plan representation, the latent form, the staircase schedule, or any individual training objective. Neither evaluation establishes an end-to-end latency advantage. Controlled ablations and timing measurements are required to answer those questions.

Summary. The two levels of evaluation characterize both planning-text quality and downstream execution. Together with the event-structured data in Sec. 3, the results support event-grounded planning as a useful interface between high-level instructions and action generation, while leaving a measurable gap to kimi3 in the reported offline text comparison.

7 Discussion and Conclusion

We introduced X-PLANNER, a task-planning front-end that connects event-grounded supervision to downstream action generation. Its Ego, UMI, and teleoperation data use a shared hierarchy with source-dependent annotation depth. Takeover-time annotations and human-designed failures supervise ongoing error recognition, with designed failures intended to reduce dependence on a single policy’s error distribution. The shared VLM backbone exposes both explicit event descriptions and continuous latent CoT representations, with Staircase Decoding reducing serialization within the latent planner. Offline two-step planning evaluation yields BERTScore-F1 of 0.9011 without baseline rescaling and a Doubao Judge Overall rating of 1.411, placing X-PLANNER above Qwen and Doubao and below kimi3. In the reported real-robot evaluation, the complete event-mode system achieves the highest average Task Progress among the evaluated methods on both reasoning and generalization suites.

Events as the interface between control and agents. Per-event textual plans provide a natural bridge between high-level agents and embodied control. X-PLANNER organizes this interface around task instructions, event states, and downstream vision-action execution. The discrete form gives a human or upstream agent an interpretable representation for inspecting or revising a plan, while the downstream model remains responsible for realizing it physically.

Decoupling interpretability and computation. Embodied reasoning must balance interpretability, which favors explicit plans, against compact computation, which favors continuous representations. X-PLANNER exposes both interfaces: the discrete form provides readable event states, while latent-to-text reconstruction gives the continuous form a textual semantic anchor. The staircase schedule shortens the planner’s serial decoding path, although any end-to-end latency benefit depends on the complete execution and scheduling stack.

Limitations and outlook. The evaluation covers offline planning text and tabletop bimanual execution with the complete event-mode system. Offline results are aggregate text and judge scores, and do not establish physical success or statistical significance. Training histories and optional initial plans are annotation-derived, whereas deployment conditions on rolling model outputs; rollout-aligned supervision is needed to narrow this oracle-context gap. Episode-end labels mark synchronized media boundaries and do not independently establish task success. Controlled comparisons of the two plan forms, end-to-end timing, and evaluation on mobile or humanoid platforms remain future work. A further direction is to replace offline event discovery with a streaming process that can revise plans online. Within these limits, X-PLANNER provides a modular planning interface for embodied foundation models.

Appendix

A Event-State Construction and Supervision Profiles

The discrete planning interface uses available visual observations, a task instruction, optional context, and a compact JSON target. Annotation levels follow Sec. 3.1: Ego and UMI provide L3 Task, L2 Subtask, and L1 Action; teleoperation additionally provides L0 Segment. Action is an executable unit, and Segment is a finer temporal span. Action-only profiles apply across sources; Segment-only and joint Action–Segment profiles require L0 annotations. Unavailable levels are omitted from supervision rather than treated as negative labels.

The three state families are summarized in Table 1. Initial-plan targets expose global event order. Ongoing targets encode task and event progress, current and next units, and continuation state; takeover-time annotations and human-designed failure examples additionally supervise error recognition. The current and next units retain their prediction-1 and prediction-2 roles at event boundaries. Episode-end targets use the final available observation and close the recorded event sequence. All assistant targets are serialized as compact JSON and supervised in full. Error annotations provide evidence for recognizing execution failures; they do not establish an automatic takeover or recovery policy.

B Context Variants and Frozen Snapshots

Prompts combine observations and a task instruction with optional initial-plan, long-history, and short-history context. Initial-plan examples contain no history. Ongoing and episode-end examples include no-context, annotation-derived, perturbed, and initial-plan-augmented variants. Perturbations alter contextual progress, ordering, or captions while preserving the observation anchor and target. These context perturbations are distinct from the human-performed failure demonstrations described in Sec. 3.2.

Training examples are materialized offline into an immutable, indexed JSONL snapshot. Each row stores visual references, prompt fields, and a complete assistant target. The DataLoader reads and tokenizes these rows without rescanning source annotations or rebuilding prompts. Each physical row is sampled once per epoch before distributed padding; category-balanced replacement is not used.

Training histories and initial plans are annotation-derived, whereas deployment consumes rolling model predictions. Context perturbations do not eliminate this oracle-context gap. Episode-end markers follow recorded-media boundaries and do not independently certify task success. The evaluation in Sec. 6 reports aggregate planning and execution quality; it does not isolate the effect of error supervision or quantify a reduction in policy-specific data bias.

References

- [1] Shuanghao Bai, Jing Lyu, Wanqi Zhou, Zhe Li, Dakai Wang, Lei Xing, Xiaoguang Zhao, Pengwei Wang, Zhongyuan Wang, Cheng Chi, et al. Latent reasoning vla: Latent thinking and prediction for vision-language-action models. *arXiv preprint arXiv:2602.01166*, 2026.
- [2] Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] Qingwen Bu, Jisong Cai, Li Chen, Xiuqi Cui, Yan Ding, Siyuan Feng, Shenyuan Gao, Xindong He, Xuan Hu, Xu Huang, et al. Agibot world colosseo: A large-scale manipulation platform for scalable and intelligent embodied systems. *arXiv preprint arXiv:2503.06669*, 2025.
- [5] Chilam Cheang, Sijin Chen, Zhongren Cui, Yingdong Hu, Liquan Huang, Tao Kong, Hang Li, Yifeng Li, Yuxiao Liu, Xiao Ma, et al. Gr-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025.
- [6] Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Sharan Narang. Think before you speak: Training language models with pause tokens. In *International Conference on Learning Representations*, 2024.
- [7] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4d: Around the world in 3,000 hours of egocentric video. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 18995–19012, 2022.
- [8] Shibo Hao, Sainbayar Gu, Haotian Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Training large language models to reason in a continuous latent space. *arXiv preprint arXiv:2412.06769*, 2024.
- [9] Xintong Hu, Xuhong Huang, Jinyu Zhang, Yutong Yao, Yuchong Sun, Qiuyue Wang, Mingsheng Li, Sicheng Xie, Yitao Liu, Junhao Chen, et al. Finevla: Fine-grained instruction alignment for steerable vision-language-action policies. *arXiv preprint arXiv:2605.27284*, 2026.
- [10] Yuhang Huang, Jiazhao Zhang, Shilong Zou, Xinwang Liu, Ruizhen Hu, and Kai Xu. Ladi-wm: A latent diffusion-based world model for predictive manipulation. *arXiv preprint arXiv:2505.11528*, 2025.
- [11] Physical Intelligence, Kevin Black, Noah Brown, James Darpinian, Karan Dhabalia, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, et al. $\pi_{0.5}$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [12] Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Nicklas Majamaki, Navdeep Jaitly, Yi-An Ma, and Lianhui Qin. Ladir: Latent diffusion enhances llms for text reasoning. *arXiv preprint arXiv:2510.04573*, 2025.
- [13] Haoqiang Kang, Yizhe Zhang, Nikki Lijing Kuang, Yi-An Ma, and Lianhui Qin. Beyond mode elicitation: Diversity-preserving reinforcement learning via latent diffusion reasoner. *arXiv preprint arXiv:2602.01705*, 2026.
- [14] Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- [15] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [16] Lin Li, Qihang Zhang, Yiming Luo, Shuai Yang, Ruilin Wang, Fei Han, Mingrui Yu, Zelin Gao, Nan Xue, Xing Zhu, et al. Causal world modeling for robot control. *arXiv preprint arXiv:2601.21998*, 2026.
- [17] Qixiu Li, Yaobo Liang, Zeyu Wang, Lin Luo, Xi Chen, Mozheng Liao, Fangyun Wei, Yu Deng, Sicheng Xu, Yizhong Zhang, et al. Cogact: A foundational vision-language-action model for synergizing cognition and action in robotic manipulation. *arXiv preprint arXiv:2411.19650*, 2024.
- [18] Zhuoyang Liu, Jiaming Liu, Hao Chen, Jiale Yu, Ziyu Guo, Chengkai Hou, Chenyang Gu, Xiangju Mi, Renrui Zhang, Kun Wu, Zhengping Che, Jian Tang, Pheng-Ann Heng, and Shanghang Zhang. Last σ : Latent spatio-temporal chain-of-thought for robotic vision-language-action model. *arXiv preprint arXiv:2601.05248*, 2026.
- [19] Yuechen Luo, Fang Li, Shaoqing Xu, Yang Ji, Zehan Zhang, Bing Wang, Yuannan Shen, Jianwei Cui, Long Chen, Guang Chen, Hangjun Ye, Zhi-Xin Yang, and Fuxi Wen. Last-vla: Thinking in latent spatio-temporal space for vision-language-action in autonomous driving. *arXiv preprint arXiv:2603.01928*, 2026.
- [20] Kepan Nan, Rui Xie, Penghao Zhou, Tiehan Fan, Zhenheng Yang, Zhijie Chen, Xiang Li, Jian Yang, and Ying Tai. Openvid-1m: A large-scale high-quality dataset for text-to-video generation. In *International Conference on Learning Representations*, 2025.
- [21] Mike Zheng Shou, Stan Weixian Lei, Weiyao Wang, Deepti Ghadiyaram, and Matt Feiszli. Generic event boundary detection: A benchmark for event segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 8075–8084, 2021.

- [22] Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- [23] Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. <https://qwen.ai/blog?id=qwen3.5>.
- [24] An Yang, Baosong Yang, Beichen Zhang, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. <https://arxiv.org/abs/2412.15115>.
- [25] Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- [26] Michał Zawalski, William Chen, Karl Pertsch, Oier Mees, Chelsea Finn, and Sergey Levine. Robotic control via embodied chain-of-thought reasoning. In *Conference on Robot Learning (CoRL)*, 2024.
- [27] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations*, 2020. <https://openreview.net/forum?id=SkeHuCVFDr>.
- [28] Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, et al. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1702–1713, 2025.
- [29] Zhide Zhong, Junfeng Li, Junjie He, Haodong Yan, Xin Gong, Guanyi Zhao, Yingjie Cai, Jiantao Gao, Xu Yan, Bingbing Liu, Yingcong Chen, Liuqing Yang, and Haoang Li. Dualcot-vla: Visual-linguistic chain of thought via parallel reasoning for vision-language-action models. *arXiv preprint arXiv:2603.22280*, 2026.
- [30] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.